

Rapport de TD n°3 : Initiation à l'algorithmique

Derrez Lorenzo

September 23, 2026

Exercice 0 :

Exercice 1 :

Hypothèses : 1 opération = 10^{-9} s (une machine à 1 milliard d'opérations par seconde). Pour une entrée de $n = 96$ octets, un algorithme linéaire effectue n opérations et un algorithme exponentiel effectue 2^n opérations.

Pour le 1^{er} cas :

- Complexité linéaire $\implies$ proportionnel à la taille en octets de l'entrée.
- Après calcul on obtient en temps linéaire 96×10^{-9} s $\approx 0,1$ μ s.

Pour le 2^e cas :

- Complexité exponentielle $\implies 2^{96}$ opérations.
- Calcul mental avec $2^{10} \approx 10^3$: $2^{96} = 2^6 \times (2^{10})^9 \approx 64 \times 10^{27} \approx 6 \times 10^{28}$ opérations.
- Temps : $2^{96} \times 10^{-9}$ s $\approx 8 \times 10^{19}$ s.
- Avec 1 an $\approx 3 \times 10^7$ s, on obtient en temps exponentiel environ $2,5 \times 10^{12}$ ans, soit environ 2 500 milliards d'années.

Exercice 2 :

Question 1

Listing 1: Complétion de la fonction exo 2

```
1 def produit (A : Matrix[integer], B : Matrix[integer]) -> Matrix[integer] :  
2   (h,w) = A;shape  
3   C = newMatrix(h, h, 0)  
4   for i in range(h) :  
5       for j in range(h) :  
6           for k in range(h) :  
7               C[i, j] = C[i, j] + A[i, k] * B[k, j]  
8   return C
```

Question 2

Sur les entrées : $A, B \rightarrow h^2$ (car $w = h$ par hypothèse)

On obtient : $O(h^2)$

Question 3

3 boucles imbriquées $\implies O(h^3)$

Question 4

C'est cubique en h.

Question 5

C occupe une place $h^2 \implies O(h^2)$ (ici i, j et k sont en $O(1)$)

Question 6

C'est quadratique en h.

Question 7

On pourrait réduire la complexité en espace en modifiant directement la valeur de la matrice A ligne par ligne plutôt qu'en créant une nouvelle matrice C.

Question 8

En cherchant sur WWW, il est dit qu'il existe un autre algorithme, celui de **Strassen**. Ce dernier implique : $O(h^{2,807})$.

Exercice 3 :

Question 1 : Arbre des appels

Question 2

- Nombre d'appels récurifs :
- Puis, on obtient le tableau suivant selon les types :

Exercice 4 :

Exercice 5 :

Question 1

Listing 2: fonction *longueurMaximalePlateau*

```
1 def longueurMaximalePlateau(T: Array[integer]) -> integer :  
2     int n = T.size  
3     int max = 0  
4     for i in range(0, n) :  
5         int j = i + 1  
6         while (j <= n and estPlateau(i, j, T)) :  
7             j = j + 1  
8             # T[i:j-1] est le plus grand plateau commençant en i  
9             if (j - 1 - i > max) :  
10                 max = j - 1 - i  
11     return max
```

- **Complexité en temps :**

3 imbrications en lien avec n la taille du tableau $\implies O(n^3)$ (car *estPlateau* est linéaire en n)

- **Complexité en espace :**

toutes les opérations sont constantes $\implies O(1)$.

Question 2

- Il faut un exemple avec une chaine maximale qui n'est pas au début, et qui n'est pas à la fin non plus.
- On peut prendre comme exemple : $T = [1, 1, 3, 3, 3, 2, 2, 7]$
- Un taille de tableau égale à 5 serait surement possible, par exemple : $T = [1, 3, 3, 3, 7]$

Question 3

Listing 3: Complétion du modèle de la fonction *IMP*

```
1  def lmp (T : Array[integer]) -> integer :
2      int lm = 0
3      int current = 0
4      for i in range(0, T.size) :
5          if i > 0 and T[i] == T[i-1] :
6              current = current + 1
7          else :
8              current = 1
9          if current > lm :
10             lm = current
11     return lm
```